

8.1 数据集的预处理

8.1.1 特征的规约与编码

在机器学习中，对数据集的特征进行编码和缩放是预处理步骤中非常重要的一部分。这些步骤有助于提升模型的性能，特别是当数据集中的特征具有不同的量纲或包含分类变量时。以下是几种常见的方法，包括使用 `sklearn` 库来实现它们。

1. 特征编码

独热编码 (One-Hot Encoding)

独热编码通常用于将分类数据（如性别、国家等）转换为机器学习算法能够处理的数值形式。

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

# 示例数据
data = np.array([[ 'male' ], [ 'female' ], [ 'male' ], [ 'female' ]]).reshape(-1, 1)

# 初始化编码器
encoder = OneHotEncoder(sparse=False)

# 拟合并转换数据
encoded_data = encoder.fit_transform(data)

print(encoded_data)
```

标签编码 (Label Encoding)

标签编码将每个类别映射到一个从0开始的整数。注意，它假设类别之间存在某种顺序，这在很多情况下并不成立。

```
from sklearn.preprocessing import LabelEncoder

# 示例数据
labels = ['male', 'female', 'male', 'female']

# 初始化编码器
encoder = LabelEncoder()

# 拟合并转换数据
encoded_labels = encoder.fit_transform(labels)

print(encoded_labels)
```

2. 特征缩放

标准化 (Standardization)

标准化是将特征缩放到均值为0，标准差为1的过程。这对于很多基于距离的算法（如K-近邻、K-均值聚类等）特别有用。

```
from sklearn.preprocessing import StandardScaler
import numpy as np

# 示例数据
data = np.array([[1, 2], [3, 4], [5, 6]])

# 初始化标准化器
scaler = StandardScaler()

# 拟合并转换数据
scaled_data = scaler.fit_transform(data)

print(scaled_data)
```

归一化 (Normalization)

归一化是将特征缩放到一个小的特定区间，通常是[0, 1]。这在很多算法中也是有用的，特别是当不同特征的取值范围差异很大时。

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# 示例数据
data = np.array([[1, 2], [3, 4], [5, 6]])

# 初始化归一化器
scaler = MinMaxScaler()

# 拟合并转换数据
scaled_data = scaler.fit_transform(data)

print(scaled_data)
```

总结

以上是使用 `sklearn` 库对数据集中的特征进行编码和缩放的一些常见方法。根据你的具体需求和数据集的特性，你可以选择最适合你的方法。注意，对于不同的算法和数据集，预处理步骤的选择可能会有所不同。

8.1.2 数据集切分与交叉验证

在 `sklearn`（`scikit-learn`）中，对数据集进行划分以及进行交叉验证是模型评估过程中非常常见的步骤。这些数据集划分方法帮助我们更好地了解模型在未见数据上的性能。

数据集划分

对于数据集的划分，我们通常使用 `train_test_split` 函数从 `sklearn.model_selection` 模块。这个函数允许我们将数据集分割成训练集和测试集，并且可以指定测试集的比例或大小。

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X = iris.data
y = iris.target

# 划分数据集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 现在X_train, y_train是训练集, X_test, y_test是测试集
```

在上面的代码中, `test_size=0.3` 表示测试集占整个数据集的30%, `random_state` 是一个随机种子, 用于确保每次划分的结果都是一样的。

交叉验证

交叉验证是一种评估统计分析方法, 它重复地划分数据为训练集和测试集, 每次使用不同的划分, 并计算平均性能指标。在 `sklearn` 中, 有几种交叉验证的策略。

1. K折交叉验证 (K-Fold Cross-Validation)

使用 `KFold` 或 `cross_val_score` 来进行K折交叉验证。这里以 `cross_val_score` 为例, 因为它直接返回了每次迭代的评分。

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

# 假设你已经有了X_train, y_train
model = LogisticRegression()

# 执行5折交叉验证
scores = cross_val_score(model, X_train, y_train, cv=5)

print(scores) # 每一折的评分
print(scores.mean()) # 所有折的平均评分
```

2. 留一交叉验证 (Leave-One-Out Cross-Validation, LOOCV)

当数据集很小时, 可以使用留一交叉验证, 它每次留一个样本作为测试集, 其余作为训练集。

```
from sklearn.model_selection import LeaveOneOut
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
loo = LeaveOneOut()

# 这里需要自己编写循环来拟合和评分
scores = []
for train_index, test_index in loo.split(X_train):
    X_train_cv, X_test_cv = X_train[train_index], X_train[test_index]
    y_train_cv, y_test_cv = y_train[train_index], y_train[test_index]
    model.fit(X_train_cv, y_train_cv)
    score = model.score(X_test_cv, y_test_cv)
    scores.append(score)

print(scores)
print(np.mean(scores))
```

注意： `cross_val_score` 函数也可以直接使用 `cv=LeaveOneOut()` 来实现留一交叉验证，但上面的例子展示了如何手动进行这一过程，以便更好地理解其背后的逻辑。

总结

`sklearn` 提供了灵活的工具来划分数据集和进行交叉验证，这对于评估机器学习模型的性能至关重要。通过 `train_test_split` 可以轻松地将数据集划分为训练集和测试集，而 `cross_val_score` 和其他交叉验证工具则可以帮助我们更全面地评估模型在不同数据子集上的表现。

8.1.3 特征选择方法

在 `sklearn`（`scikit-learn`）中，特征选择与特征降维是机器学习过程中的重要步骤，它们可以帮助减少模型的复杂度，提高模型的泛化能力，并提升模型的性能。以下是关于如何在 `sklearn` 中进行特征选择与特征降维的详细解释：

特征选择

特征选择是从原始特征集中选择出对模型预测最有用的特征子集的过程。`sklearn` 提供了多种特征选择的方法，包括过滤方法、包装方法和嵌入方法。

过滤方法 (Filter Methods)

过滤方法使用统计学或信息论方法来评估每个特征的相关性，并选择最相关的特征。常用的过滤方法包括：

1. 方差选择法 (Variance Threshold) :

- 移除所有方差低于某个阈值的特征。这种方法假设方差小的特征可能是不重要的或冗余的。
- 在 `sklearn` 中，可以使用 `VarianceThreshold` 类来实现。

2. 互信息法 (Mutual Information) :

- 评估每个特征和目标变量之间的互信息，并选择具有高互信息的特征。
- 在 `sklearn` 中，可以使用 `mutual_info_classif` (分类问题) 和 `mutual_info_regression` (回归问题) 函数来计算互信息，并使用 `SelectKBest` 类来选择特征。

包装方法 (Wrapper Methods)

包装方法使用模型的性能作为特征选择的指标，并尝试找到最优的特征子集。常用的包装方法包括：

• 递归特征消除 (Recursive Feature Elimination, RFE) :

- 从初始的特征集开始，使用模型对特征进行排序，并删除最不重要的特征，直到达到所需的特征数量。
- 在 `sklearn` 中，可以使用 `RFE` 或 `RFECV` (带有交叉验证的RFE) 类来实现。

嵌入方法 (Embedded Methods)

嵌入方法将特征选择作为模型训练过程的一部分，并在学习过程中选择最优的特征子集。常用的嵌入方法包括：

• 基于L1正则化的方法:

- 使用L1范数作为正则化项，对模型参数进行惩罚，从而降低模型复杂度并选择有用的特征。
- 在 `sklearn` 中，可以使用 `Lasso` 回归模型来实现L1正则化，并通过选择具有非零系数的特征来进行特征选择。

• 基于树的方法:

- 决策树和随机森林等基于树的方法可以在训练过程中自动评估特征的重要性，并可以用于特征选择。
- 在 `sklearn` 中，可以使用 `feature_importances_` 属性来获取特征的重要性，或使用 `SelectFromModel` 类结合基于树的模型来进行特征选择。

当然可以。以下是针对几种常见的特征选择方法的代码用法和代码案例，这些示例使用了 `sklearn` 库。

1. 方差选择法 (Variance Threshold)

```
from sklearn.feature_selection import VarianceThreshold
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X, y = iris.data, iris.target

# 应用方差选择法，移除方差小于0.8的特征
selector = VarianceThreshold(threshold=0.8)
X_transformed = selector.fit_transform(X)

# 查看变换后的特征数量
print(X_transformed.shape)
```

注意：这里的 threshold=0.8 是示例值，实际应用中应根据数据特性调整。

2. 互信息法 (使用SelectKBest)

```
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X, y = iris.data, iris.target

# 使用互信息法选择最好的两个特征
selector = SelectKBest(mutual_info_classif, k=2)
X_new = selector.fit_transform(X, y)

# 查看选择的特征索引
print(selector.get_support(indices=True))

# 查看变换后的特征
print(X_new.shape)
```

3. 递归特征消除 (RFE)

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X, y = iris.data, iris.target

# 使用逻辑回归作为基模型，递归特征消除选择5个特征
estimator = LogisticRegression()
selector = RFE(estimator, 5, step=1)
X_new = selector.fit_transform(X, y)

# 查看选择的特征索引
print(selector.support_)
# 查看变换后的特征
print(X_new.shape)
```

4. 基于L1正则化的特征选择 (使用Lasso)

```
from sklearn.linear_model import Lasso
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X, y = iris.data, iris.target

# 使用Lasso回归并选择alpha参数
lasso = Lasso(alpha=0.1)
lasso.fit(X, y)

# 查看哪些特征的系数非零，即选择了哪些特征
print(lasso.coef_ != 0)

# 注意：这里通常不会直接转换X，而是基于coef_的结果来选择特征
# 例如，可以创建一个新的DataFrame或数组，只包含coef_非零对应的特征
```


5. 嵌入方法（以随机森林为例）

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel
from sklearn.datasets import load_iris

# 加载数据集
iris = load_iris()
X, y = iris.data, iris.target

# 使用随机森林作为基模型
clf = RandomForestClassifier(n_estimators=10)
clf.fit(X, y)

# 使用SelectFromModel选择重要性高于平均的特征
selector = SelectFromModel(clf, prefit=True, threshold='mean')
X_new = selector.transform(X)

# 查看选择的特征数量
print(X_new.shape)

# 查看哪些特征被选择了
print(selector.get_support())
```

注意：以上代码中的 `threshold` 参数在 `SelectFromModel` 中根据基模型的不同可能有所不同，这里使用 `'mean'` 作为示例，意味着选择重要性高于平均的特征。在实际应用中，你可能需要根据模型的 `feature_importances_` 来调整这个阈值。

特征降维

特征降维是将高维特征空间转换为低维特征空间的过程，以减少数据的复杂性和计算成本。sklearn 提供了多种特征降维的方法，包括主成分分析（PCA）和线性判别分析（LDA）等。

主成分分析（PCA）

PCA是一种常用的线性降维方法，它通过保留数据中的主要特征（即方差最大的方向）来降低数据的维度。在 sklearn 中，可以使用 `PCA` 类来实现PCA降维。

- 优点：
 - 能够有效降低数据的维度，同时保留数据中的大部分信息。
 - 可以去除数据中的噪声和冗余信息。

- **缺点：**

- 是一种线性降维方法，对于非线性关系的数据可能效果不佳。
- 降维后的特征可能难以解释。

线性判别分析 (LDA)

LDA是一种有监督的降维方法，它试图找到一种线性组合，使得类内方差最小化而类间方差最大化。然而，需要注意的是，在 `sklearn` 中，LDA通常用于分类任务（通过 `LinearDiscriminantAnalysis` 类），并且它本身并不直接用于降维（尽管在分类过程中会隐含地进行特征变换）。对于降维目的，更常使用的是PCA或其他无监督的降维方法。